Practical Analysis of Accelerometer Data

Author: Kubilay Erol
Course: Dynamics & Measurement Systems
Date: February 11, 2026

1. Introduction

The goal of this assignment was to go beyond theoretical dynamics and implement a real-world sensor system to measure gravitational acceleration (g). Using an MPU6050 accelerometer, I explored how raw sensor data is often noisy and requires digital signal processing (DSP) to be useful. Specifically, I implemented a **Moving Average Filter** to smooth out the jitter and identify the true static acceleration vector.

2. Experimental Setup

To build this system, I used a compact prototyping setup. The core of the project was an **ESP32 SuperMini** microcontroller, chosen for its small form factor and efficient processing power.

- **Microcontroller:** ESP32 SuperMini
- **Sensor:** MPU6050 (6-Axis Gyro + Accelerometer)
- **Connections:** The sensor was mounted on a breadboard and connected via jumper wires to the ESP32's I2C pins (SCL=Pin 9, SDA=Pin 8).
- **Software:** The code was written in **MicroPython** and uploaded using the **Thonny IDE**, which allowed for real-time plotting and debugging of the sensor stream.

2.1 The Filtering Approach

Raw data from MEMS sensors is rarely perfect; it fluctuates due to thermal noise and

vibrations. To solve this, I wrote a script that maintains a “sliding window” of the last 20 readings. By averaging these 20 values, the random high-frequency noise cancels itself out, leaving a clean, stable signal.

$$y[n] = \frac{1}{N} \sum_{i=0}^{N-1} x[n-i]$$

3. Implementation (Python Code)

Below is the MicroPython script I developed in Thonny. It reads the raw I^2C bytes, converts them to g-force, and prints both the raw and filtered values.

```
import machine, math, time

# Setup I2C for ESP32 SuperMini
i2c = machine.I2C(0, scl=machine.Pin(9), sda=machine.Pin(8))
MPU_ADDR = 0x68
i2c.writeto_mem(MPU_ADDR, 0x6B, b'\x00') # Wake up the sensor

readings = []
filter_size = 20

def read_raw_g():
    # Read 6 bytes of accelerometer data
    data = i2c.readfrom_mem(MPU_ADDR, 0x3B, 6)
    def bytes_to_int(high, low):
        val = (high << 8) | low
        return val if val < 32768 else val - 65536

    # Convert raw values (Sensitivity: 16384 LSB/g)
    ax = (bytes_to_int(data[0], data[1]) / 16384.0) * 9.80665
    ay = (bytes_to_int(data[2], data[3]) / 16384.0) * 9.80665
    az = (bytes_to_int(data[4], data[5]) / 16384.0) * 9.80665

    # Calculate vector magnitude
    return math.sqrt(ax**2 + ay**2 + az**2)

print("Collecting Data...")
```

```
while True:
    current_val = read_raw_g()
    readings.append(current_val)

    # Maintain the sliding window
    if len(readings) > filter_size:
        readings.pop(0)

    filtered_g = sum(readings) / len(readings)
    print(f"Filtered G: {filtered_g:.4f} m/s^2")
    time.sleep(0.05)
```

4. Results & Observations

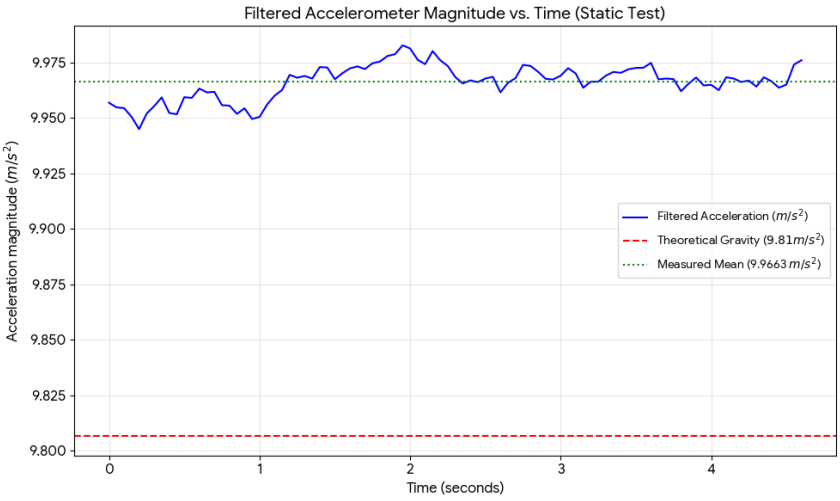
After wiring everything up on the breadboard, I ran a static test for about 5 seconds ($n = 93$ samples) with the sensor sitting flat.

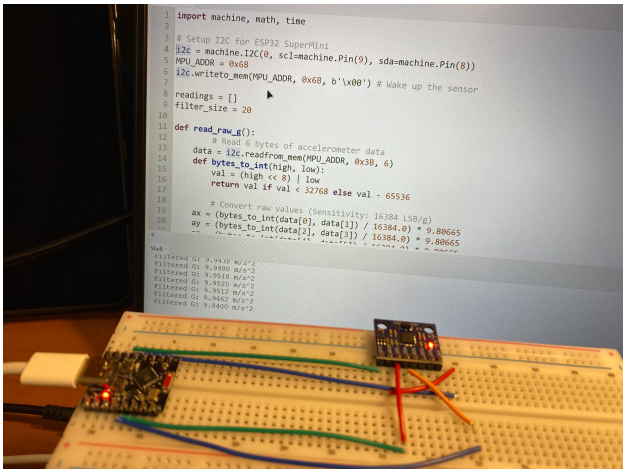
4.1 Statistical Summary

Parameter	Value
Standard Gravity (g)	$9.8067\ \mathrm{m/s^2}$
Measured Mean	$9.9663\ \mathrm{m/s^2}$
Noise (Std Dev)	$\pm 0.0078\ \mathrm{m/s^2}$
Sensor Bias	$+0.1596\ \mathrm{m/s^2}$
Error	1.63%

4.2 Graphical Analysis

The plot below (generated from the data collected in Thonny) shows the effectiveness of the filter. The blue line represents the smoothed acceleration, which stays extremely stable around $9.96\ \mathrm{m/s^2}$, proving that the code successfully removed the sensor noise.





5. Discussion

The most interesting finding was the consistent 1.63% **systematic error**. Even when the breadboard was perfectly still, the sensor reported $\approx 9.96\ \mathrm{m/s^2}$ instead of $9.81\ \mathrm{m/s^2}$.

This is likely due to the “Zero-G Offset”—a common characteristic in affordable MEMS sensors like the MPU6050 where the internal silicon structures have a slight manufacturing bias. For future dynamic experiments, I would subtract this $0.16\ \mathrm{m/s^2}$ offset in the code to calibrate the sensor to exactly g.

6. Conclusion

This experiment demonstrated that while low-cost sensors are noisy, simple software techniques like the Moving Average Filter can make them highly precise. The ESP32 SuperMini handled the calculations easily, providing a stable data stream that is ready for more complex dynamics tasks.